1016

langchain > 6. agent > 5. TavilySearch

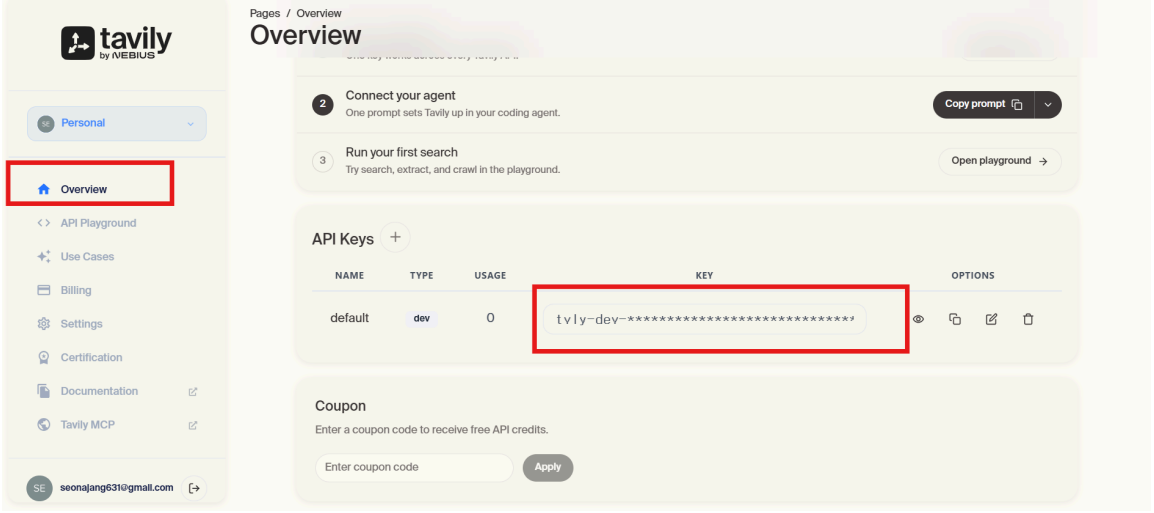
TavilySearch: 실시간 웹 검색을 가능하게 하는 tool

(가능하다면 크롤링 코드 짜는 것을 추천)

[Setup]

1. https://www.tavily.com/

   → 구글 계정으로 로그인 후 API key 발급



   → .env 파일에 api key 추가

[Tool 생성]

TavilySearch

```python
from langchain_tavily import TavilySearch

search_new = TavilySearch(
    max_results=3,
    topic="general",              # 또는 "news", "finance" 등
    include_answer=True,          # 답변 포함 여부
    include_raw_content=False,    # 원본 내용 포함 여부
    include_images=False,         # 이미지 포함 여부
    search_depth="advanced",        # "basic" 또는 "advanced"
    include_domains=[
        "https://news.naver.com/section/105",
        "https://news.daum.net/"
    ],
    exclude_domains=None           # 필요하면 제외 도메인 지정 가능
)
```

✓ 0.0s

[Tool 실행: .invoke]

```python
result_new = search_new.invoke("최신 삼성전자 뉴스들 알려줘.") #→ "입력하는 텍스트 -> query"
```

✓ 1.0s

[Agent가 사용할 tool list]

tools

```python
tools = [summarize_news, search_weather_info, search_stock_info]
```

[AI Agent에 tool 붙혀줌]

Agent

```python
from langchain.agents import create_agent

agent = create_agent(
    model=llm,
    tools=tools,
    system_prompt="""
    당신은 주어진 도구를 반드시 사용해서만 답변해야 하는 AI 어시스턴트입니다.
    절대로 자신의 지식으로 직접 답변하지 마세요.
    모든 최종 답변은 반드시 도구의 출력 결과를 기반으로 해야 합니다.
    """
)
```

[AI Agent 호출]

뉴스 검색

```python
from langchain_core.messages import HumanMessage

user_input = "삼성전자 관련 최신 뉴스를 요약해주세요."
result1 = agent.invoke(
    {"messages": HumanMessage(content=user_input)}
)
```

[48]

→ tools 중 필요한 tool을 Agent가 알아서 호출

langchain > 6. agent > 4-1 & 4-2

- QuerySQLDatabaseTool : Tool 정의할 때 사람이 직접 쿼리 입력

```python
@tool
def search_employee_by_name(name: str) -> str:
    """이름으로 직원을 검색합니다.
        - name: 직원명
    """
    try:
        query = f"""
        SELECT e.name, e.department, e.salary, e.hire_date, d.manager, d.budget
        FROM employees e
        LEFT JOIN departments d ON e.department = d.dept_name
        WHERE e.name LIKE '%{name}%'
        """
        result = sql_tool.invoke(query)
        if result:
            return f"'{name}' 검색 결과:\n{ast.literal_eval(result)}"
        else:
            return f"'{name}'과 일치하는 직원을 찾을 수 없습니다."
    except Exception as e:
        return f"직원 검색 중 오류가 발생했습니다: {str(e)}"
```

- SQLDatabaseToolkit : AI가 쿼리까지 작성

6단계: SQLDatabaseToolkit 생성

```python
from langchain_community.agent_toolkits.sql.toolkit import SQLDatabaseToolkit

toolkit = SQLDatabaseToolkit(db=db, llm=llm)
```

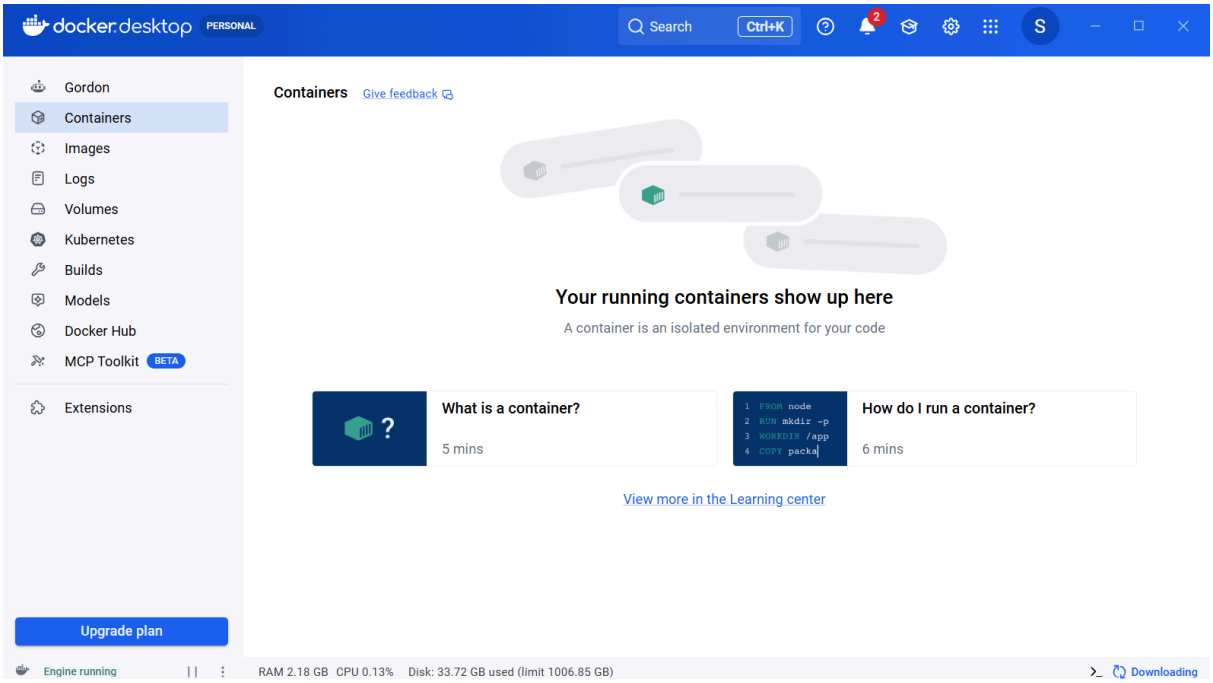
```python
tools = toolkit.get_tools()

for tool in tools:
    print("=" * 50)
    print(f"Tool 이름: {tool.name}")
    print(f"Tool 설명: {tool.description}")
    print(f"Tool 인풋 파라미터: {tool.args}")
    print(f"Tool return_direct: {tool.return_direct}")
```

langchain > 6. agent > 5. MySQL with SQLDatabaseToolkit

Chinook mysql에 저장하기

1. docker 열기 (기존에 있던 것들 다 삭제해서 container에 아무것도 없게 만들기)



2. vs 코드로 돌아가서 terminal에서 다음 코드 실행

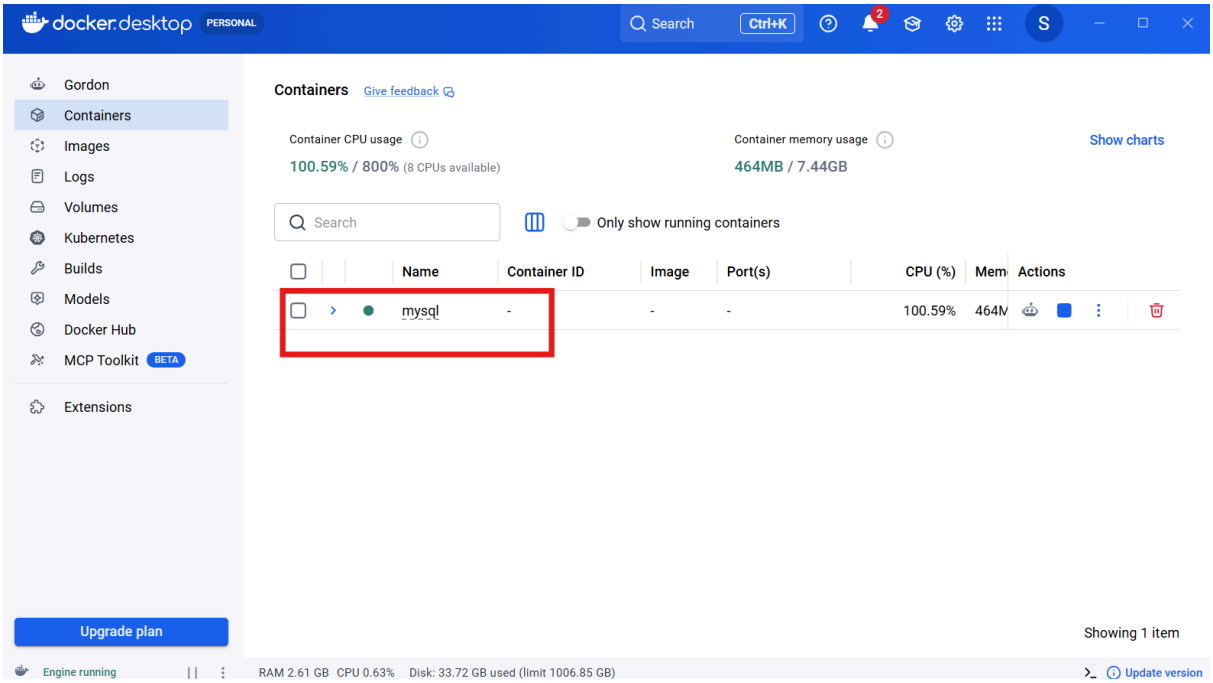
```
(6-agent) PS C:\dev\study\langchain\6. agent> cd .\mysql\
(6-agent) PS C:\dev\study\langchain\6. agent\mysql> ls


    디렉터리: C:\dev\study\langchain\6. agent\mysql

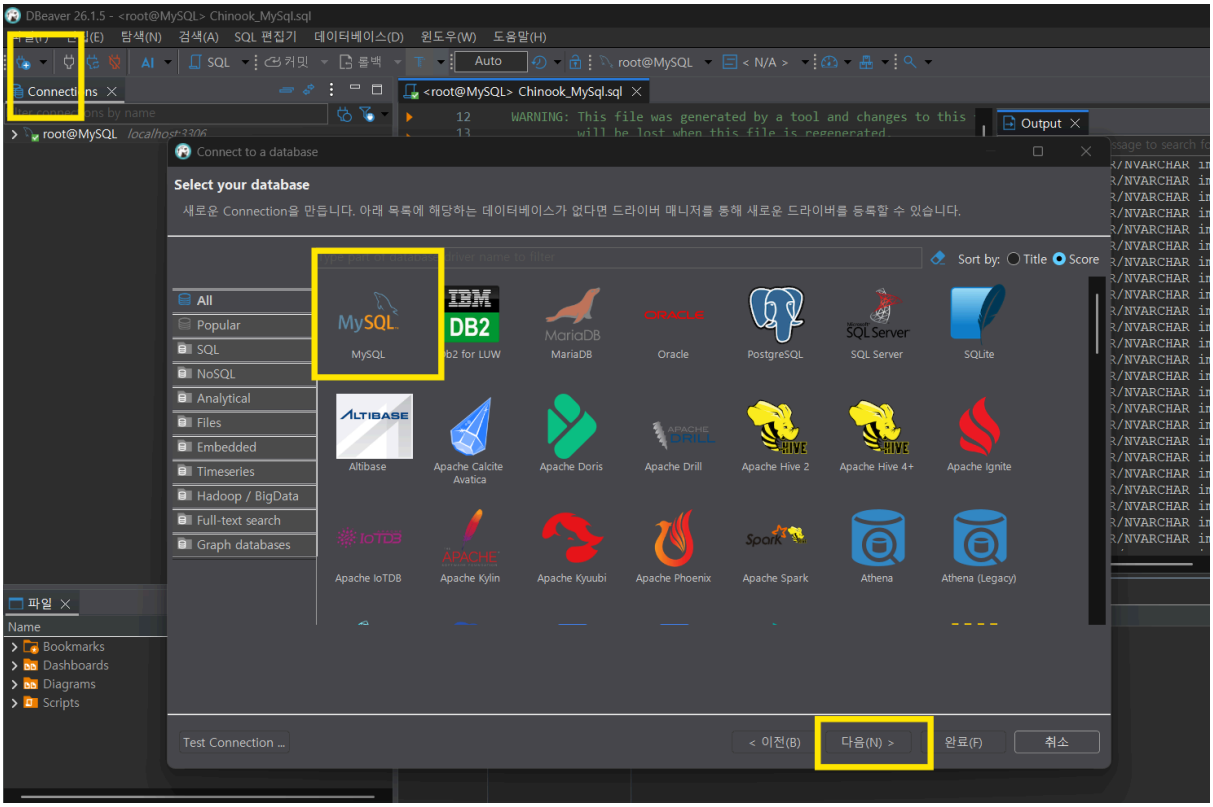

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        2026-10-14  오전 11:26         616450 Chinook_MySql.sql
-a----        2026-10-14  오전 11:26            406 docker-compose.yml


(6-agent) PS C:\dev\study\langchain\6. agent\mysql> docker-compose up -d
```

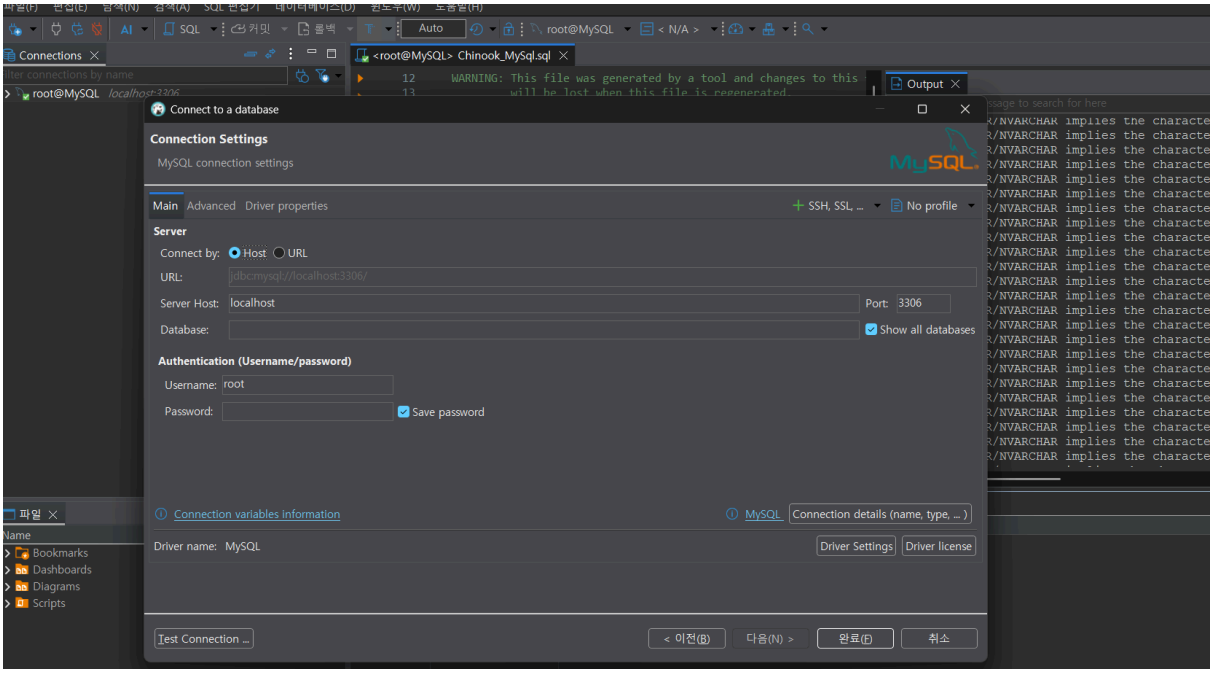
3. 실행하면 도커에 다음과 같이 mysql 생김

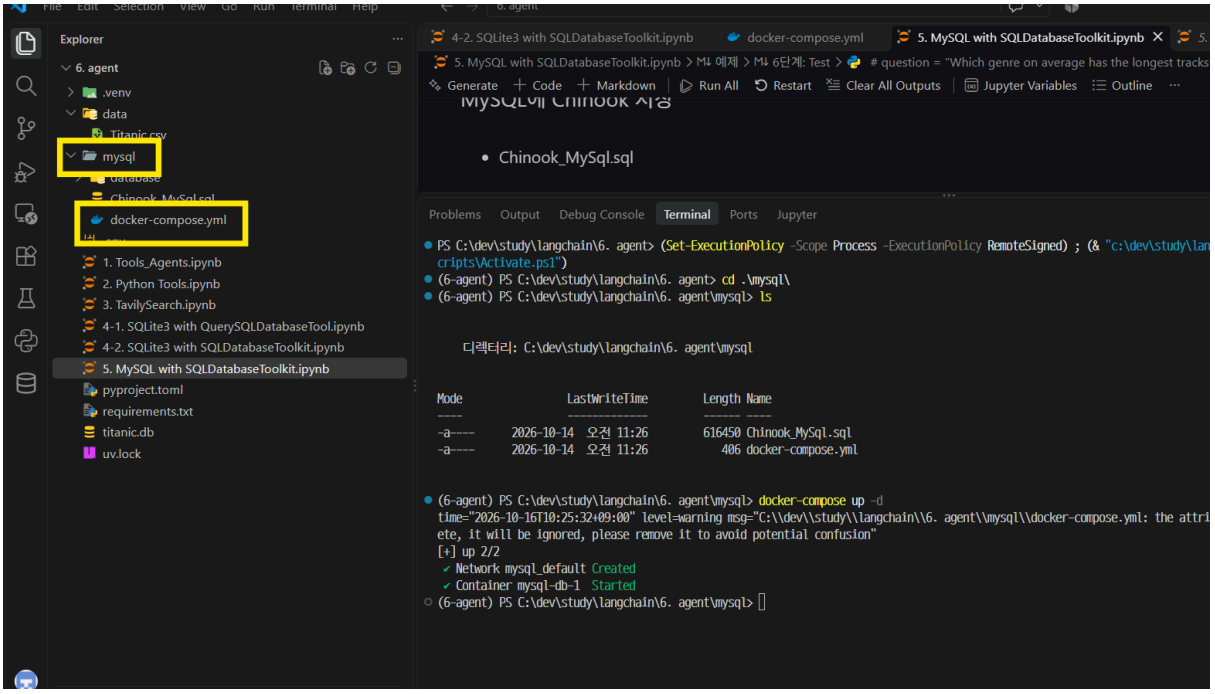


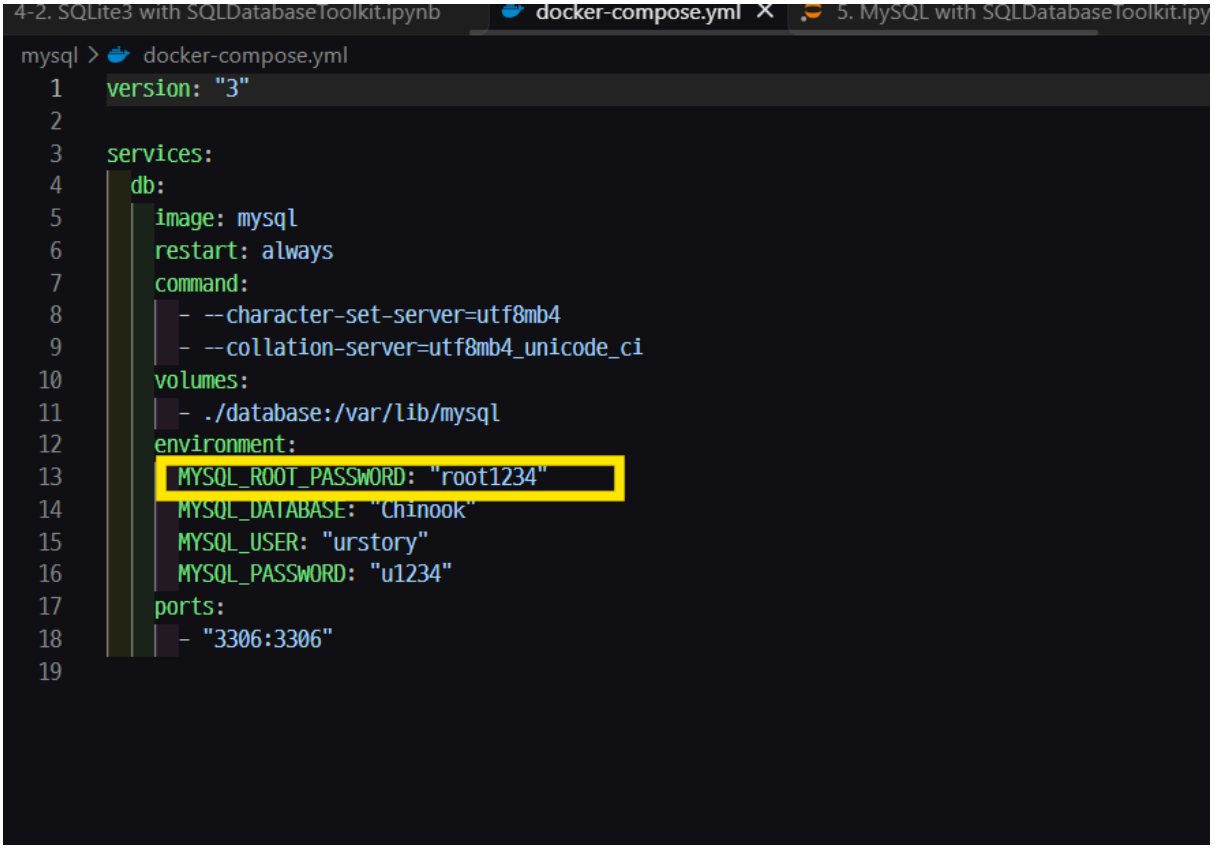
4. dbeaver 들어가기



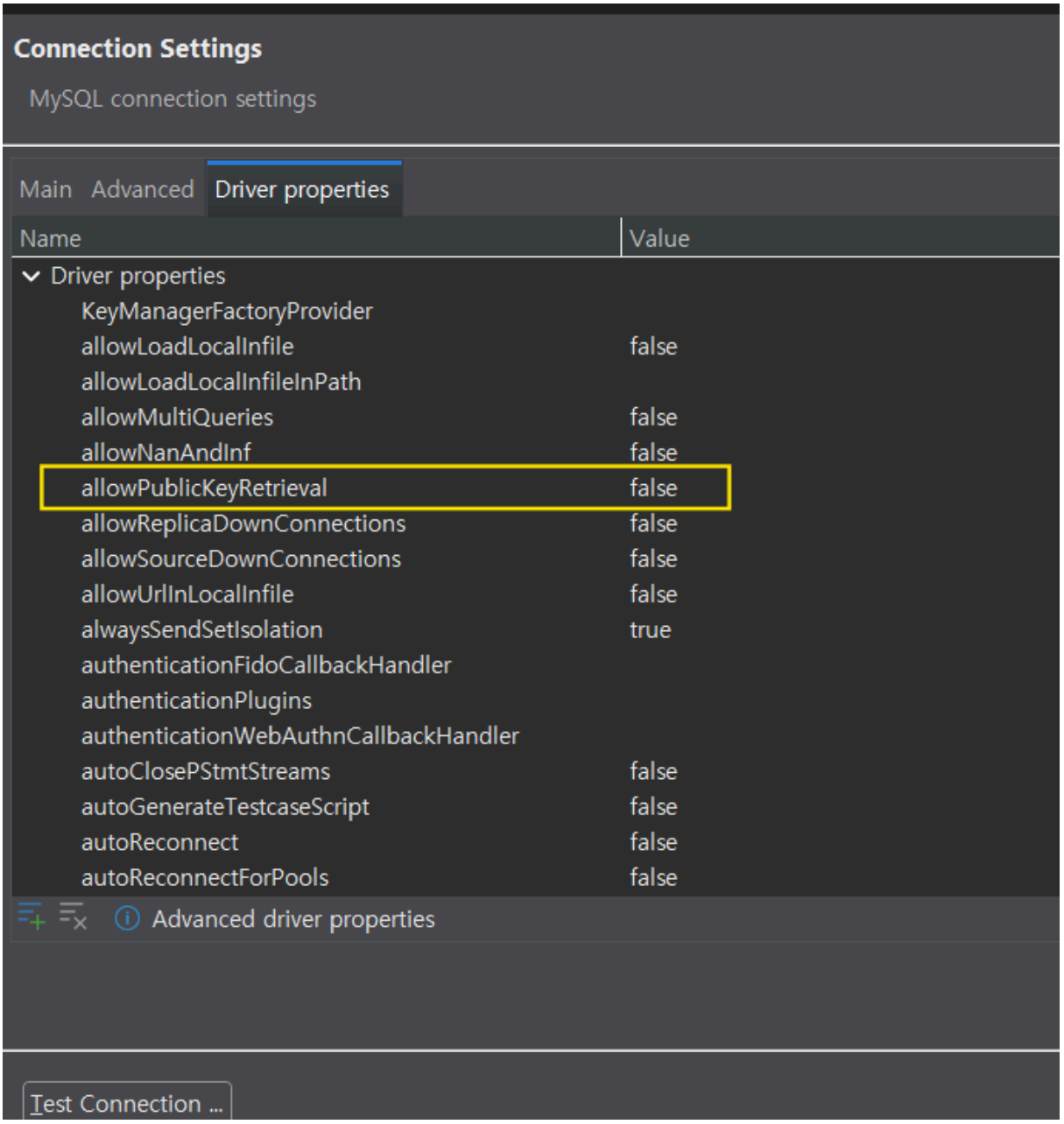
5. yaml 파일에서 패스워드 받아와서 입력


Connections
root@MySQL localhost:3306
<root@MySQL> Chinook_MySql.sql
WARNING: This file was generated by a tool and changes to this
will be lost when this file is regenerated.
Output
Connect to a database
Connection Settings
MySQL connection settings
Main Advanced Driver properties
SSH, SSL, ...
No profile
Server
Connect by: Host URL
URL: jdbc:mysql://localhost:3306/
Server Host: localhost
Port: 3306
Database:
Show all databases
Authentication (Username/password)
Username: root
Password:
Save password
Connection variables information
MySQL Connection details (name, type, ...)
Driver name: MySQL
Driver Settings
Driver license
Test Connection ...
< 이전(B)
다음(N) >
완료(F)
취소
파일
Name
Bookmarks
Dashboards
Diagrams
Scripts


Explorer
6. agent
.venv
data
Titanic.csv
mysql
docker-compose.yml
1. Tools_Agents.ipynb
2. Python Tools.ipynb
3. TavilySearch.ipynb
4-1. SQLite3 with QuerySQLDatabaseTool.ipynb
4-2. SQLite3 with SQLDatabaseToolkit.ipynb
5. MySQL with SQLDatabaseToolkit.ipynb
pyproject.toml
requirements.txt
titanic.db
uv.lock
4-2. SQLite3 with SQLDatabaseToolkit.ipynb
docker-compose.yml
5. MySQL with SQLDatabaseToolkit.ipynb
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Chinook_MySql.sql
Problems
Output
Debug Console
Terminal
Ports
Jupyter
(6-agent) PS C:\dev\study\langchain\6. agent\mysql> ls
디렉터리: C:\dev\study\langchain\6. agent\mysql
Mode LastWriteTime Length Name
-a---- 2026-10-14 오전 11:26 616450 Chinook_MySql.sql
-a---- 2026-10-14 오전 11:26 406 docker-compose.yml
(6-agent) PS C:\dev\study\langchain\6. agent\mysql> docker-compose up -d
[+] up 2/2
Network mysql_default Created
Container mysql-db-1 Started
(6-agent) PS C:\dev\study\langchain\6. agent\mysql>



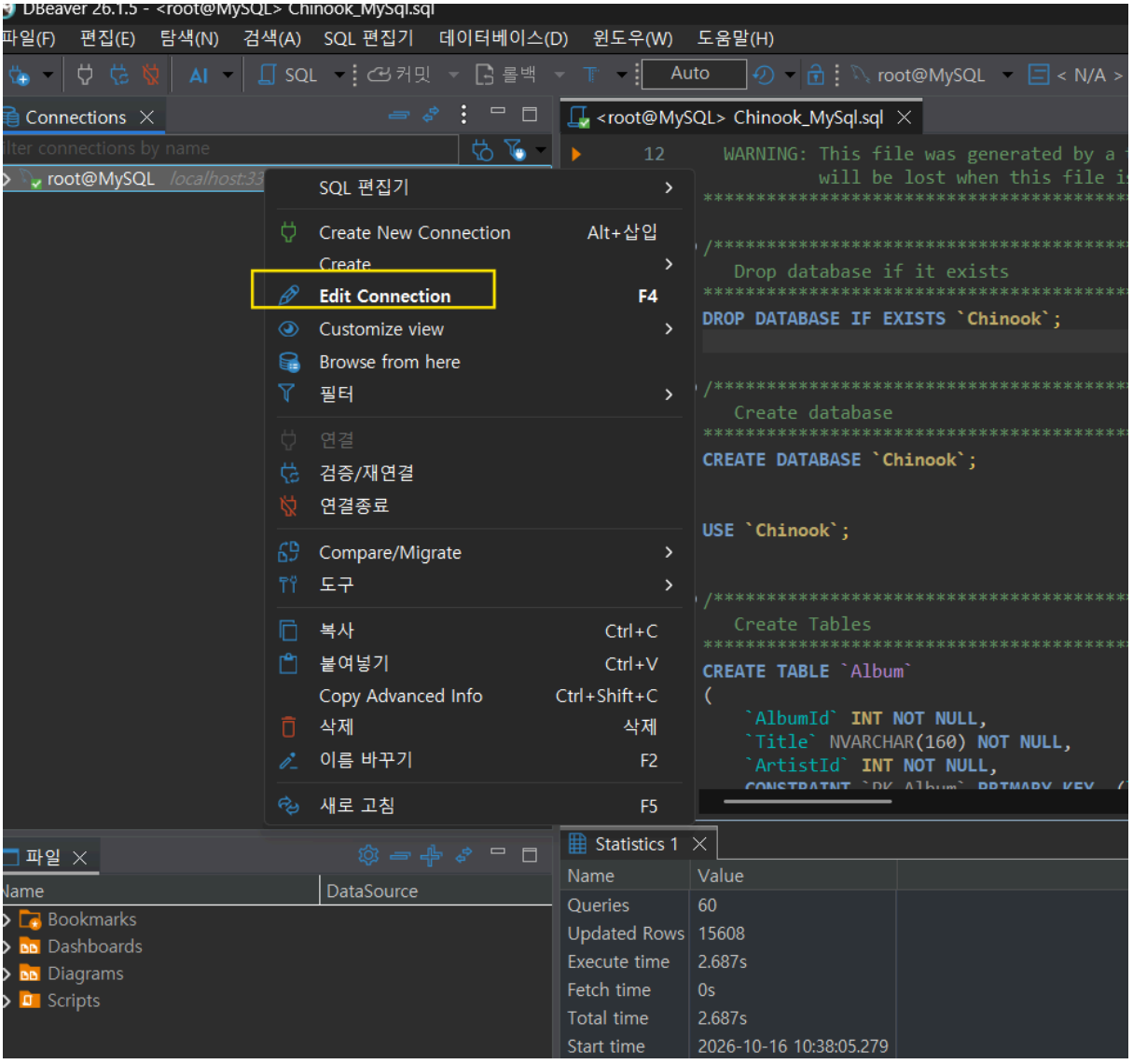
→ docker password 칸에 root1234 치고



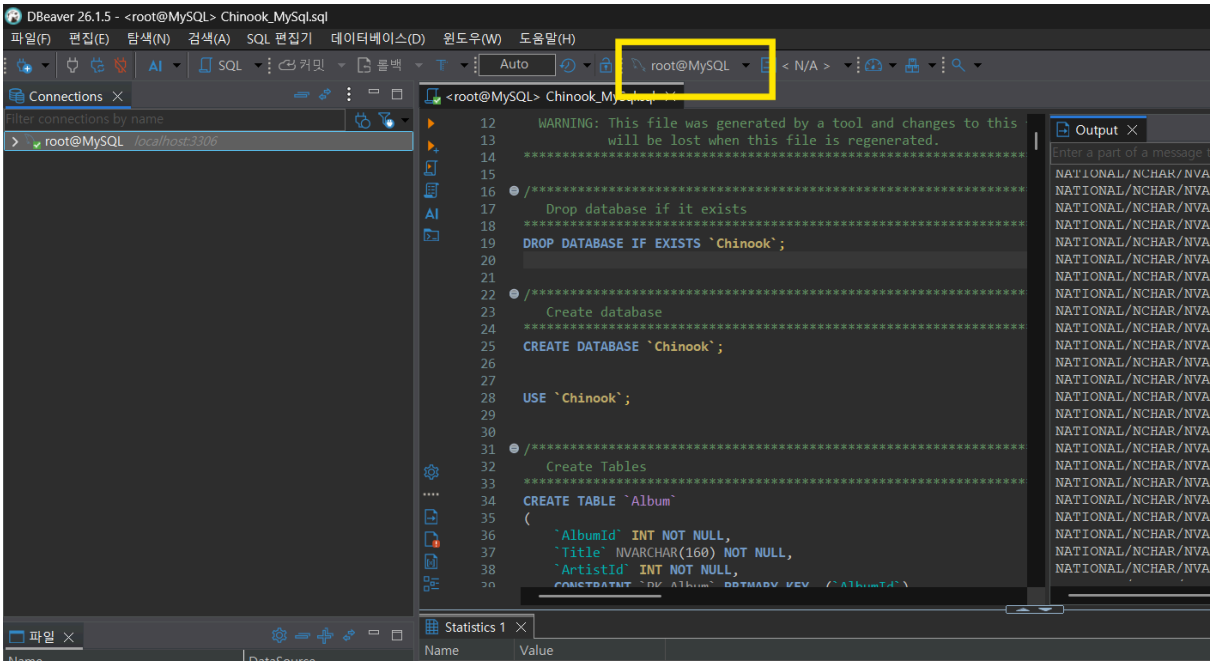
driver properties에서 노란색 박스 친 부분 true로 바꾸기

그러면 root 계정으로 db 생성 됨 (기존에 연결되어 있던 db랑 헷갈릴 수 있으니 dbeaver에 있는 db 다 삭제하고 해보세여...)

6.

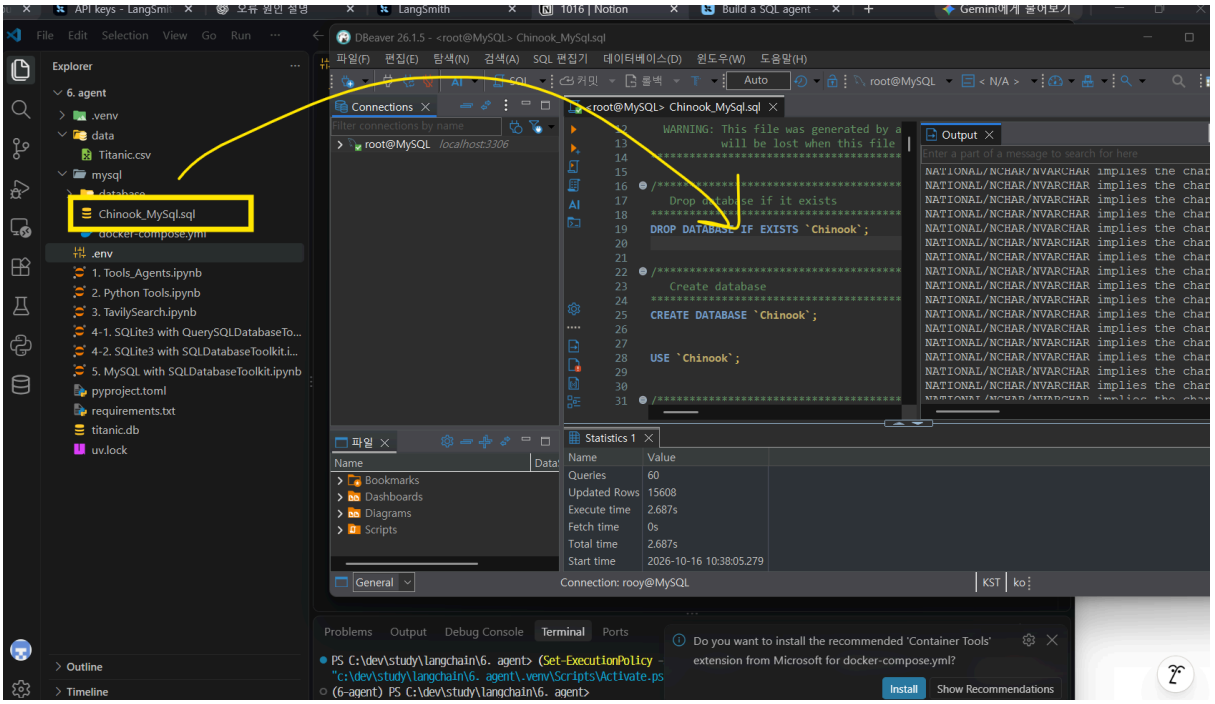


마우스 우클릭하여 edit connection 들어가서 connection 이름을 root@MySQL로 바꿈 (이름 형식 → root: 계정 종류 [user 계정이랑 구분됨] @ db 종류)

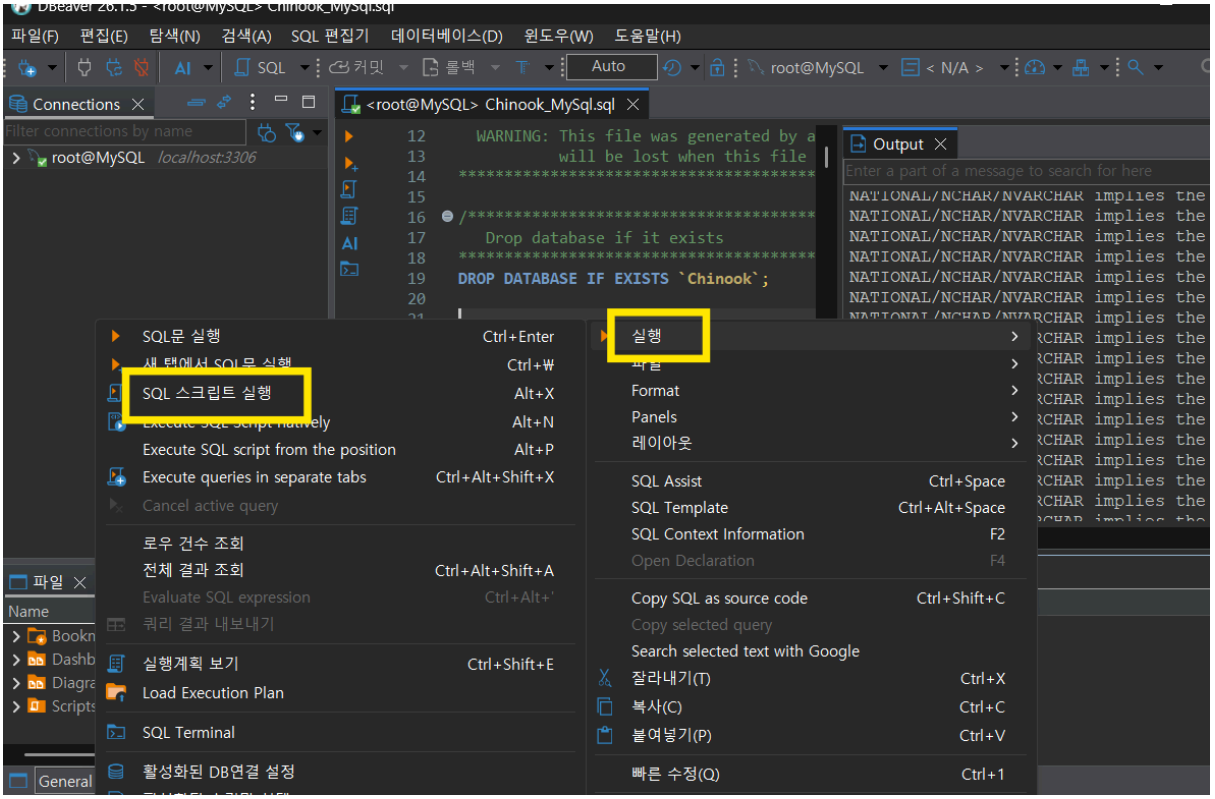


→ 노란 박스 친 부분에서 connection으로 root@MYSQL 선택

7. chinook_MySql 파일 드래그 해서 dbeaver에 떨어뜨리기



8.



8. 다시 vs 코드로 돌아와서 1단계부터 코드 실행

LangGraph

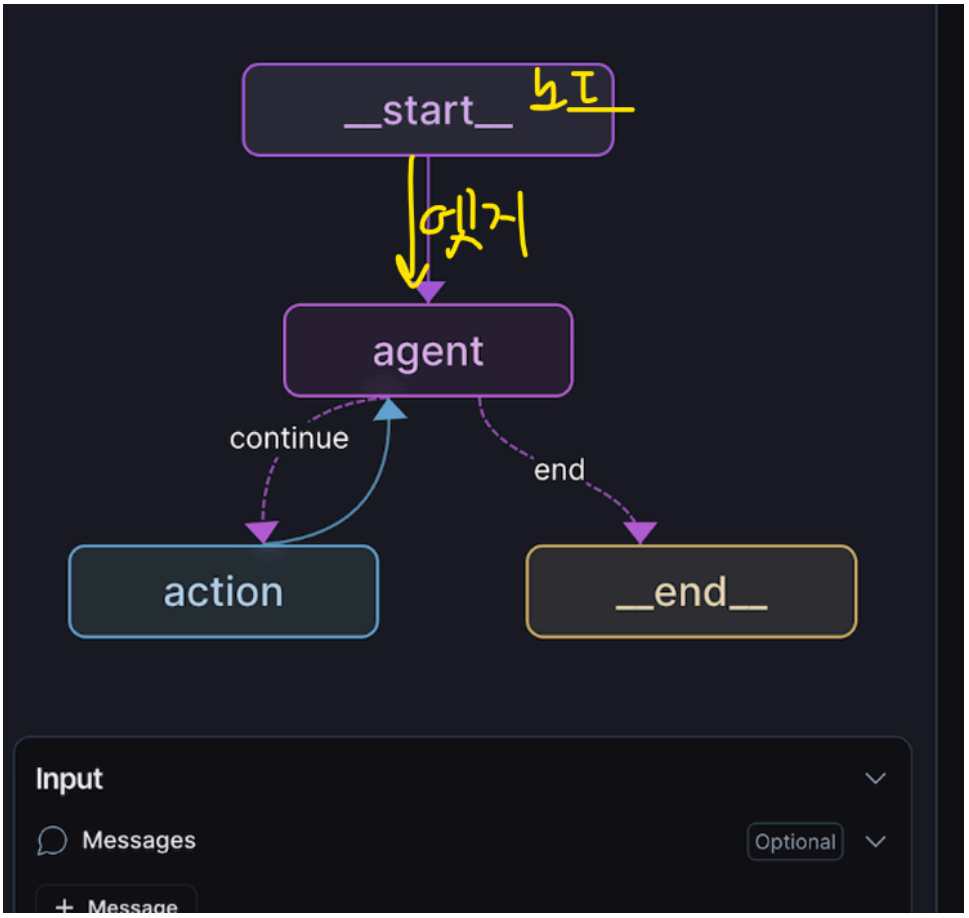
- langchain의 서비스 중 하나
- multi agent를 구현하는 도구 중 하나
- langchain만으로는 복잡한 비즈니스 로직 구현하기 어려울 수 있음

  → 복잡한 체인이나 에이전트를 노드와 엣지 (그래프)로 시각화

  → 이를 통해 분기, 반복, 병렬 처리 등 제어 흐름을 명시적으로 표현

  [그래프]

  - 노드: 함수/기능/액션
  - 엣지: 연결



langchain (선형적인 로직)

langgraph는 그래프 기반 구조를 따르기 때문에 복잡한 구조를 구현할 수 있다. (실행 순서가 langchain처럼 정해져 있지 않음) → 협력 agent 만들고 싶을 때는 langgraph